

UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS



FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS DE INFORMACIÓN

TRABAJO FINAL - Hito 2

CURSO: PROGRAMACIÓN ORIENTADA A OBJETOS

CASO: 12 - SISTEMA DE GESTIÓN DE CONSULTORIO MÉDICO

DOCENTE: JOSE LUIS OJEDA MERINO

NCR: 12564

GRUPO: N° 2

ESTUDIANTES:

Mercado Torres, Jayron Fabricio (U20251K060)

Dario Francisco Rodriguez Quispe (U202523240)

Salcedo Ramírez, Yasuri Yamile (U20251J944)

Aldave Ocospoma Diego Ricardo (U20251K043)

SEMESTRE ACADÉMICO: 2026 - 01

2026

Lima - Perú

1. Resumen

El presente documento describe el desarrollo de un sistema de gestión para consultorios médicos, diseñado como una solución orientada a optimizar y automatizar los procesos operativos y administrativos de los centros de salud. El sistema tiene como objetivo principal gestionar de manera eficiente el registro centralizado de pacientes y médicos, la programación y validación de citas médicas, el control de asistencia y la generación de reportes operativos relacionados con la atención brindada.

Asimismo, este proyecto refleja la importancia de la ingeniería de sistemas de la información, en el desarrollo de soluciones informáticas capaces de analizar, organizar y procesar información de manera eficiente. Según Ortiz Zambrano, Sangacha Tapia y Alarcón Santillán (2017), los ingenieros de sistemas computacionales tienen como objetivo desarrollar los procesos relacionados con los sistemas informáticos de manera eficaz, a través del uso de técnicas que le permitan analizar y procesar la información, en función de la gestión de proyectos informáticos con un alto nivel de profesionalidad. Para el desarrollo de este sistema se utilizó el paradigma de **Programación Orientada a Objetos (POO)** en Python. La estructura del programa se organizó a través de clases y objetos para representar las entidades principales del consultorio y gestionar sus procesos de manera eficiente. Este enfoque garantiza una solución informática funcional, ordenada y perfectamente adaptable a las necesidades de gestión del centro de salud.

Índice

1.	Resumen.....	1
2.	Objetivos del estudiante.....	3
3.	Capítulo 1: Presentación y sustentación del problema a resolver	4
3.1.	Tema del trabajo.....	5
3.2.	Sustento teórico.....	5
3.3.	Motivación por la que escogió el tema.....	6
4.	Capítulo 2: Estructura de archivos.....	6
4.1.	Estructura del archivo excel (CSV).....	7
4.2.	Diagrama de clases.....	7
4.3.	Plan de actividades	8
5.	Capítulo 3: Listado de funcionalidades cumplidas y Manual de Usuario.....	9
5.1.	Requerimientos Funcionales	9
5.2.	Análisis de Datos	9
5.3.	Diseño del menú e interfaces de entrada.....	20
5.3.	Diseño de reportes	27
5.4.	Manual de Usuario.....	34
5.5.	Estándar de nomenclatura.....	41
6.	Conclusiones.....	41
7.	Recomendaciones	43
8.	Glosario.....	44
9.	Bibliografía	44
10.	Participación.....	45

2. Objetivos del estudiante

Aldave Ocrospoma Diego Ricardo:

Aplicar los conocimientos básicos de programación en Python para desarrollar un Sistema de Gestión de Consultorio Médico, que nos permita registrar información de pacientes, médicos y citas para fortalecer conocimientos de lógica de programación.

Mercado Torres, Jayron Fabricio:

Poner en práctica el conocimiento adquirido en el curso de POO, para lograr desarrollar un Sistema de Gestión de Consultorio Médico, que nos permite registrar pacientes y médicos, gestionar citas y controlar la asistencia y estados.

Rodriguez Quispe, Dario Francisco:

El desarrollo del Sistema de Gestión de Consultorio Médico me permitió fortalecer mis habilidades de programación en Python, especialmente en el uso de funciones, listas y diccionarios, POO, y manejo de datos para organizar información.

Además, el trabajo en equipo contribuyó al desarrollo de capacidades de análisis, resolución de problemas y diseño lógico de soluciones informáticas orientadas a necesidades reales.

Salcedo Ramírez, Yasuri Yamile:

El desarrollo del Sistema de Gestión de Consultorio Médico permitió aplicar conocimientos de programación para diseñar una solución orientada al registro y control de pacientes, médicos y citas médicas. Asimismo, el proyecto contribuyó al cumplimiento del Student Outcome 2 al desarrollar un programa funcional relacionado con la atención y bienestar de los pacientes.

3. Capítulo 1: Presentación y sustentación del problema a resolver

3.1. Tema del trabajo

Desarrollo de un Sistema de Gestión de Consultorio Médico utilizando Python.

Objetivo:

El objetivo de nuestro sistema de gestión es desarrollar una aplicación en Python que nos permita registrar pacientes y médicos, gestionar citas y controlar asistencia y estados.

Alcance:

- Registrar paciente permite ingresar y validar datos personales.
- Registrar médico permite guardar datos profesionales y especialidad.
- Crear cita permite asignar médico, fecha, hora y motivo validando disponibilidad.
- Ver citas muestra todas las citas y controla la asistencia.
- Ver reportes genera atenciones, rankings e inasistencias.

3.2. Sustento teórico

Los sistemas de gestión son herramientas informáticas que permiten organizar, almacenar y administrar información de manera eficiente. En el área de la salud, estos sistemas facilitan el control de pacientes, médicos y citas médicas, ayudando a mejorar los procesos administrativos y reducir errores en el manejo de la información. Para Villafuerte Salas y colaboradores en su trabajo de información del 2020, afirman que los sistemas de información ayudan a administrar, recolectar, recuperar, procesar, almacenar y distribuir información relevante para los procesos fundamentales y las particularidades de cada organización.

El presente proyecto fue desarrollado utilizando el lenguaje de programación Python, debido a su facilidad de uso y flexibilidad para crear aplicaciones orientadas a la gestión de datos. García Monsálvez en 2017 afirma que Python es realmente un lenguaje de propósito general y ha ido ganando popularidad en varios ámbitos como el desarrollo rápido de aplicaciones web,

administración de sistemas, ciencia de datos, computación científica, inteligencia artificial, internet de las cosas, etc.

Para el desarrollo de este sistema se optó por el paradigma de **Programación Orientada a Objetos (POO)**. Bajo este enfoque, los requerimientos y la lógica del consultorio se modelaron mediante el uso de clases y objetos, permitiendo agrupar tanto las características como los comportamientos de las entidades principales en estructuras ordenadas y seguras. De igual manera, esta arquitectura facilita un código más limpio, modular y eficiente para el mantenimiento continuo del sistema de gestión médica.

3.3. Motivación por la que escogió el tema

Se escogió el desarrollo de un Sistema de Gestión de Consultorio Médico debido a la importancia de mejorar la organización y administración de la información en los centros de salud. En muchos consultorios, los registros de pacientes y citas médicas aún se realizan de manera manual, lo que puede ocasionar desorden, pérdida de información y retrasos en la atención.

Según Recharte (2023), en la actualidad las instituciones privadas y estatales tienen la necesidad de usar las tecnologías de información y comunicaciones como herramientas administrativas estratégicas que les permitan desenvolverse con eficiencia y eficacia en el desempeño de sus funciones.

Además, este proyecto permitió aplicar los conocimientos adquiridos en clase, llevando la lógica del negocio hacia un modelo basado en componentes interactivos. Asimismo, se buscó desarrollar una solución práctica y funcional que facilite el control de pacientes, médicos y citas médicas de manera más eficiente y organizada.

4. Capítulo 2: Estructura de archivos

4.1. Estructura del archivo excel (CSV)

Agenda.csv

Columna	Tipo de dato	Descripción
DNI	String	DNI del paciente.
Medico	String	Nombre del médico.
Código de medico	String	Código identificador del médico.
Fecha	String	Fecha programada para la cita.
Horario	String	Hora de la cita.
Motivo	String	Motivo de la consulta médica.
Asistencia	String	Indica si el paciente asistió.

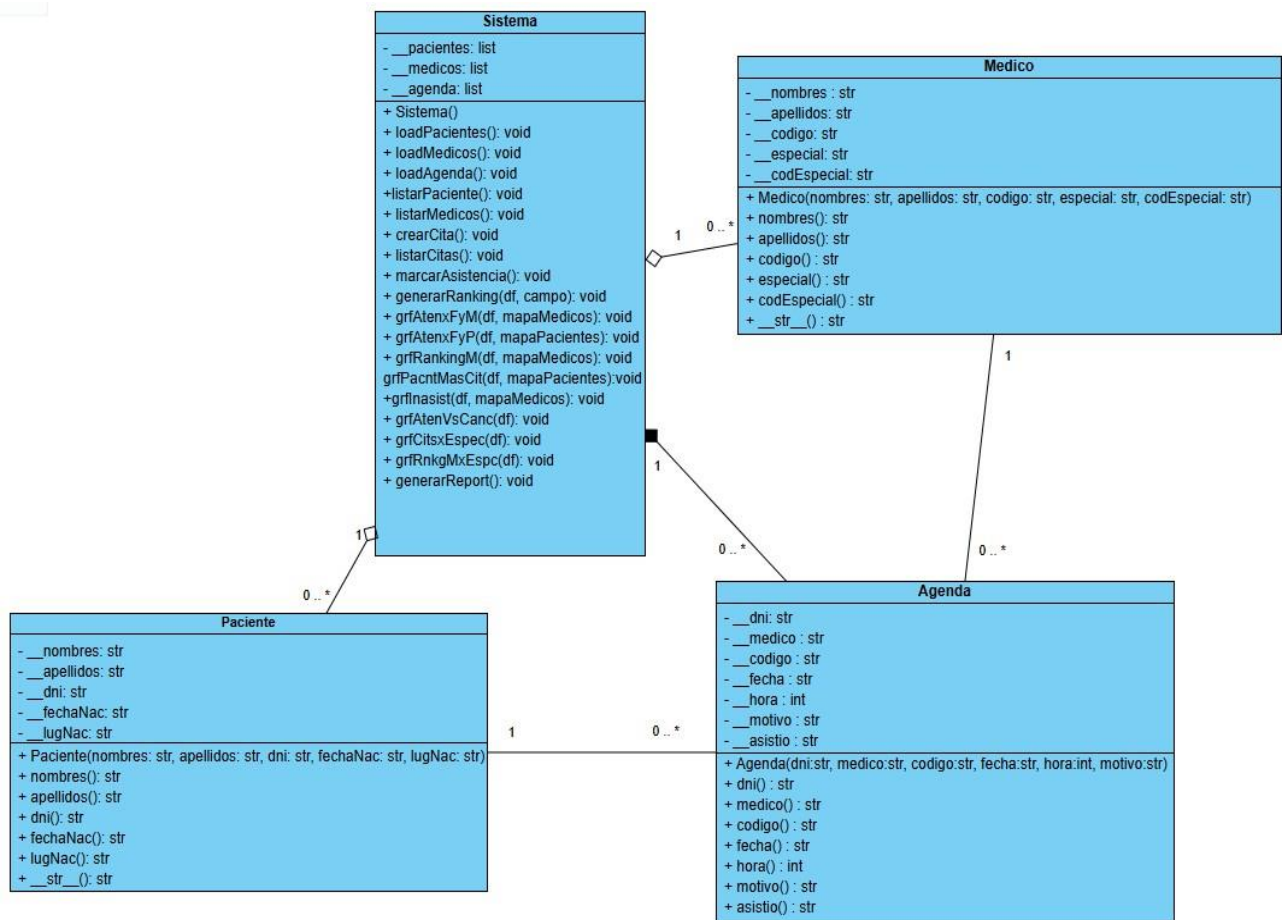
Medicos.csv

Columna	Tipo de dato	Descripción
Nombre	String	Nombre del médico.
Apellido	String	Apellidos del médico.
Código	String	Código único del médico.
Especialidad	String	Especialidad médica.
Código de especialidad	String	Código que indica la especialidad.

Pacientes.csv

Columna	Tipo de dato	Descripción
Nombres	String	Nombres del paciente.
Apellidos	String	Apellidos del paciente.
DNI	String	DNI del paciente
Fecha nacimiento	String	Fecha de nacimiento del paciente.
Lugar nacimiento	String	Lugar de nacimiento del paciente.

4.2. Diagrama de clases



4.3. Plan de actividades

Tabla 1

Plan de actividades

Integrantes:	Tarea realizada
Aldave Ocrospoma Diego Ricardo	Desarrollo de la clase Agenda, método crearCita() con control de horarios y asistencia, y apoyo en la documentación del proyecto.
Mercado Torres, Jayron Fabricio	Desarrollo de la clase Sistema, listados con Pandas y desarrollo de la clase Medico, método registrarMedico() con validaciones de colegiatura en la clase Sistema, funciones del menú (main, menu, validOpcion), además de la documentación del sistema.
Rodriguez Quispe, Dario Francisco	Desarrollo del método generarReport() junto a otros métodos complementarios del mismo, añadiendo gráficos al programa, así como apoyo en la documentación del proyecto.

Salcedo Ramírez, Yasuri Yamile	Desarrollo de la clase Paciente, método registrarPaciente() y listarPacientes() con validaciones de colegiatura en la clase Sistema, además de la documentación del proyecto.
--------------------------------	---

5. Capítulo 3: Listado de funcionalidades cumplidas y Manual de Usuario

5.1. Requerimientos Funcionales

Tabla 2:

Requerimientos funcionales del sistema

Requerimiento Funcional	Descripción	Cumplió
Almacenamiento	Guardar información de manera permanente para evitar la pérdida de datos al cerrar el programa	Cumplió totalmente
Registro de pacientes	Permite registrar y almacenar la información de los pacientes en el sistema.	Cumplió totalmente
Registro de médicos	Permite registrar y almacenar la información de los médicos, incluyendo su especialidad y código de especialidad.	Cumplió totalmente
Creación de citas	Permite programar citas médicas asociando pacientes, médicos, fecha, hora y motivo de consulta.	Cumplió totalmente
Listado de citas	Permite visualizar las citas médicas registradas en el sistema.	Cumplió totalmente
Reportes	Permite mostrar información organizada de pacientes, médicos y citas mediante reportes generados con Pandas.	Cumplió totalmente
Menú	Permite ofrecer una interfaz de navegación intuitiva basada en opciones para acceder a cada módulo de forma controlada.	Cumplió totalmente

5.2. Análisis de Datos

Tabla 3:

Análisis de Datos - Clase Paciente

Variables	Descripción	Tipo de Dato	Restricciones	Uso
__nombres	Nombres del paciente	str	No vacío, sin números, .title()	Intermedio
__apellidos	Apellidos del paciente	str	No vacío, sin números, .title()	Intermedio
__dni	Dni del paciente	str	8 dígitos, no repetidos	Intermedio
__fechaNac	Fecha de nacimiento	str	8 dígitos numéricos (DDMMAAAA)	Intermedio
__lugNac	Lugar de nacimiento	str	No vacío, sin números, .title()	Intermedio

Tabla 4:

Análisis de Datos - Clase Médico

Variables	Descripción	Tipo de Dato	Restricciones	Uso
__nombres	Nombres del médico	str	No vacío, sin números, .title()	Intermedio
__apellidos	Apellidos del médico	str	No vacío, sin números, .title()	Intermedio
__codigo	Código de colegiatura	str	6 dígitos y unico	Intermedio
__especial	Especialidad médica	str	No vacío, sin números, .title()	Intermedio
_codEspecial	Código de especialidad	str	6 dígitos numéricos o "Desconocido"	Intermedio

Tabla 5:

Análisis de Datos - Clase Agenda

Variables	Descripción	Tipo de Dato	Restricciones	Uso
-----------	-------------	--------------	---------------	-----

__dni	DNI del paciente asociado a la agenda	str	Almacena la propiedad extraída del objeto Paciente	Intermedio
__medico	Apellido del médico asignado a la cita	str	Almacena la propiedad extraída del objeto Médico	Intermedio
__codigo	Código de colegiatura del médico	str	Debe corresponder a un médico registrado y contener 6 dígitos	Intermedio
__fecha	Fecha de la cita médica programada	str	8 dígitos numéricos (DDMMAAAA)	Intermedio
__hora	Hora de cita médica programada	int	Formato militar de 0 a 23	Intermedio
__motivo	Descripción breve de la consulta	str	No vacío, sin números	Intermedio
__asistio	Control de asistencia médica	str	Estado pendiente de confirmación para cambiar “Pendiente”	Intermedio

Tabla 6:
Análisis de Datos - Clase Sistema y Variables Globales de Control

Variables	Descripción	Tipo de Dato	Restricciones	Uso
__pacientes	Lista que guarda objetos de tipo Paciente registrados en el sistema	list	Solo admite objetos de tipo Paciente. No se permiten DNIs duplicados dentro de la lista	Intermedio
__medicos	Lista que guarda objetos del tipo Medico registrados en el sistema	list	Solo admite objetos de tipo Médico; no se permiten códigos de colegiatura duplicados dentro de la lista	Intermedio
__agenda	Lista que guarda objetos del tipo Agenda registrados en el sistema	list	Solo admite objetos de tipo Agenda; no se permite que un mismo médico o paciente tenga dos citas en la misma fecha y hora	Intermedio
Método: def loadPacientes(self)				
dfP	DataFrame que almacena los datos leídos del archivo pacientes.csv	DataFrame	Debe contener las columnas Nombres, Apellidos, DNI, Fecha nacimiento y Lugar nacimiento; si el archivo no existe, se captura FileNotFoundError y no se genera	Intermedio
paciente	Objeto Paciente construido con los datos de una fila del DataFrame	Paciente	Debe crearse con los 5 datos obtenidos de la fila correspondiente (Nombres, Apellidos, DNI, Fecha nacimiento, Lugar nacimiento)	Intermedio
Método: def loadMedicos(self)				
dfM	DataFrame que almacena los	<i>DataFrame</i>	Debe contener las columnas	Intermedio

	datos leídos del archivo medicos.csv		Nombres, Apellidos, Código, Especialidad y Código especialidad; si el archivo no existe, se captura FileNotFoundError y no se genera	
medico	Objeto Médico construido con los datos de una fila del DataFrame	<i>Medico</i>	Debe crearse con los 5 datos obtenidos de la fila correspondiente (Nombres, Apellidos, Código, Especialidad, Código especialidad)	Intermedio
Método: def loadAgenda(self):				
dfA	DataFrame que almacena los datos leídos del archivo agenda.csv	DataFrame	Debe contener las columnas DNI, Médico, Código médico, Fecha, Horario, Motivo y Asistencia. Si el archivo no existe, se captura FileNotFoundError y no se genera.	Intermedio
agenda	Objeto Agenda construido con los datos de una fila del DataFrame	Agenda	Debe crearse con los 7 datos obtenidos de la fila correspondiente (DNI, Médico, Código médico, Fecha, Horario, Motivo, Asistencia)	Intermedio
Método: def registrarPaciente(self):				
nombresP	Nombres del paciente	str	No vacío, sin números	Entrada
apellidosP	Apellidos del paciente	str	No vacío, sin números	Entrada
dniP	DNI del paciente	str	8 dígitos, no repetido	Entrada
fechaNacP	Fecha de nacimiento	str	Exactamente 8 dígitos, formato DDMMAAAA	Entrada

lugNacP	Lugar de nacimiento	str	No vacío, sin números	Entrada
paciente	Objeto con los datos del paciente	Paciente(objeto)	Instancia con los datos validados	Intermedio
dictsP	Diccionario temporal del paciente	dict	Estructura de claves de texto	Intermedio
dfPacientes	DataFrame de pandas	DataFrame	Estructura bidimensional de datos	Intermedio
"pacientes.csv"	Archivo de almacenamiento externo	Archivo (CSV)	Separado por comas (,), sin índice	Salida
Método: def listarPacientes(self):				
"pacientes.csv"	Archivo físico con los datos guardados	Archivo (CSV)	Debe existir para poder leerse	Entrada
df	Tabla de datos cargada en memoria	DataFrame	Estructura de Pandas, lee todo como texto (dtype=str)	Intermedio
Método: def registrarMedico(self):				
nombresM	Almacena temporalmente los nombres ingresados del médico.	str	No puede estar vacío ni contener números.	Entrada
apellidosM	Almacena temporalmente los apellidos del médico.	str	No puede estar vacío ni contener números.	Entrada
codigoM	Almacena temporalmente el código de colegiatura.	str	Debe contener 6 dígitos y no estar repetido.	Entrada
opcionM	Guarda la respuesta del usuario sobre si posee especialidad.	str	Solo acepta "si" o "no".	Entrada
especialM	Almacena temporalmente la especialidad del médico.	str	No puede estar vacía ni contener números.	Entrada
codEspecialM	Almacena temporalmente	str	Debe contener 6 dígitos.	Entrada

	el código de la especialidad.			
medico	Objeto creado de la clase Medico.	Medico	Debe crearse con datos previamente validados.	Intermedio
listaDictsM	Lista que almacena diccionarios con los datos de los médicos.	list	Contiene únicamente diccionarios.	Intermedio
dfMedicos	DataFrame con la información de los médicos registrados.	DataFrame	Se genera a partir de listaDictsM.	Intermedio
Método: def listarMedicos(self):				
df	DataFrame que almacena la información leída del archivo medicos.csv.	DataFrame	El archivo debe existir para ser leído correctamente.	Intermedio
Método: def crearCita(self):				
dniC	Almacena el DNI ingresado por el usuario.	str	Debe existir en la lista de pacientes registrados.	Entrada
existe	Indica si el paciente fue encontrado en el sistema.	bool	Solo admite True o False.	Intermedio
opcion	Almacena la opción del médico seleccionada.	int	Debe corresponder a un médico de la lista.	Entrada
medicoC	Almacena el apellido del médico seleccionado.	str	Debe corresponder a un médico registrado.	Intermedio
codigoC	Almacena el código del médico seleccionado.	str	Debe pertenecer al médico elegido.	Intermedio
fechaC	Almacena la fecha ingresada para la cita.	str	Debe tener formato DDMMAAAA y corresponder a los años 2026 o 2027.	Entrada
medicOcupadoC	Indica si el médico ya tiene	bool	Solo admite True o False.	Intermedio

	una cita en el mismo horario.			
pacOcupadoC	Indica si el paciente ya tiene una cita en el mismo horario.	bool	Solo admite True o False.	Intermedio
motivoC	Almacena el motivo de la consulta.	str	No puede estar vacío ni contener números.	Entrada
agenda	Objeto creado de la clase Agenda.	Agenda	Debe crearse con datos previamente validados.	Intermedio
listaDictsA	Lista que almacena los diccionarios con la información de las citas.	list	Contiene únicamente diccionarios.	Intermedio
dfAgenda	DataFrame que contiene la información de todas las citas.	DataFrame	Se genera a partir de listaDictsA.	Intermedio
Método: def listarCitas(self):				
df	DataFrame que almacena la información leída del archivo agenda.csv.	DataFrame	El archivo agenda.csv debe existir.	Intermedio
Método: def marcarAsistencia(self):				
dni	Almacena el DNI ingresado por el usuario.	str	Debe existir en la lista de pacientes registrados.	Entrada
encontrado	Indica si el paciente fue hallado en el sistema.	bool	Solo admite True o False.	Intermedio
tieneCita	Indica si el paciente tiene al menos una cita registrada.	bool	Solo admite True o False.	Intermedio
cita	Objeto de la agenda recorrido en el bucle de búsqueda.	Cita	Debe pertenecer a la lista de citas registradas.	Intermedio

op	Almacena la respuesta ingresada sobre la asistencia.	str	Solo admite 'si' o 'no'.	Entrada
listaDictsA	Lista que almacena los diccionarios con la información actualizada de las citas.	list	Contiene únicamente diccionarios.	Intermedio
dictsA	Diccionario con los datos de una cita individual.	dict	Debe contener las claves DNI, Medico, Codigo medico, Fecha, Horario, Motivo y Asistencia.	Intermedio
dfAgenda	DataFrame que contiene la información actualizada de todas las citas.	DataFrame	Se genera a partir de listaDictsA.	Intermedio
Método: def grfAtenxFyM(self, df, mapaMedicos):				
atendidas	Filtra las citas donde la asistencia fue "si".	DataFrame	Solo contiene filas con Asistencia == "si".	Intermedio
conteo	Código del médico recorrido en el bucle.	str	Debe existir en la columna "Código medico".	Intermedio
datos	Subconjunto del DataFrame filtrado por código de médico.	DataFrame	Contiene solo las citas del médico actual.	Intermedio
resumen	Conteo de atenciones por fecha para un médico.	Series	Indexado por fecha.	Intermedio
nombre	Nombre del médico obtenido desde el mapa.	str	Debe existir en mapaMedicos, si no, usa el código.	Intermedio
Método: def grfAtenxFyP(self, df, mapaPacientes):				
atendidas	Filtra las citas donde la	DataFrame	Solo contiene filas con Asistencia == "si".	Intermedio

	asistencia fue "si".			
pacientesTop	Los 5 pacientes con más citas atendidas.	Index	Máximo 5 elementos	Intermedio
dni	DNI del paciente recorrido en el bucle.	str	Debe existir en la columna "DNI".	Intermedio
datos	Subconjunto del DataFrame filtrado por DNI.	DataFrame	Contiene solo las citas del paciente actual.	Intermedio
resumen	Conteo de atenciones por fecha para un paciente.	Series	Indexado por fecha.	Intermedio
nombre	Nombre del paciente obtenido desde el mapa.	str	Debe existir en mapaPacientes; si no, usa el DNI.	Intermedio
Método: def grfRankingM(self, df, mapaMedicos)				
medicOrd	Ranking de médicos ordeado por citas atendidas de mayor a menor.	Series	Generado por generarRanking().	Intermedio
nombres	Lista de nombres de médicos obtenidos desde el mapa.	list	Usa el código si el nombre no existe e mapaMedicos.	Salida
Método: def grfPacntMasCit(self, df, mapaPacientes)				
pacOrd	Top 10 pacientes con más citas atendidas.	Series	Máximo 10 elementos, generado por generarRanking().	Intermedio
nombres	Lista de nombres de pacientes obtenidos desde el mapa.	list	Usa el DNI si el nombre no existe en mapaPacientes.	Intermedio
Método: def grfInasist(self, df, mapaMedicos)				
inasistentes	Filtra las citas donde la	DataFrame	Solo contiene filas con Asistencia == "no".	Intermedio

	asistencia fue "no".			
conteo	Conteo de inasistencias por médico ordenado de mayor a menor.	Series	Indexado por código de médico.	Intermedio
nombres	Lista de nombres de médicos obtenidos desde el mapa.	list	Usa el código si el nombre no existe en mapaMedicos.	Intermedio
Método: def grfAtenVsCanc(self, df)				
atendidasN	Cantidad de citas con asistencia "si".	int	Debe ser mayor o igual a 0.	Intermedio
canceladasN	Cantidad de citas con asistencia "no".	int	Debe ser mayor o igual a 0.	Intermedio
Método: def grfCitsxEspec(self, df)				
mapaEspecialidad	Diccionario que relaciona código de médico con su especialidad.	dict	Generado desde la lista de médicos registrados.	Intermedio
conteo	Conteo de citas agrupadas por especialidad.	Series	Indexado por especialidad.	Intermedio
Método: grfRnkgMxEspc(self, df)				
especialidadesR	Lista de especialidades únicas sin repetición	list	Construida recorriendo la lista de médicos	Intermedio
medicosEsp	Lista de médicos filtrados por especialidad	list	Contiene solo médicos de la especialidad actual	Intermedio
nombres	Lista de nombres completos de los médicos	list	Formato "Nombres Apellidos"	Intermedio
cantidades	Lista con la cantidad de citas de cada médico	list	Valores enteros mayores o iguales a 0	Intermedio
contadorR	Cantidad de citas de un	int	Mayor o igual a 0	Intermedio

	médico específico			
Método: def generarReporte(self):				
listaDictE	Lista que almacena los diccionarios con la información de cada cita.	list	Contiene únicamente diccionarios.	Intermedio
dictsE	Diccionario con los datos de una cita individual.	dict	Debe contener las claves DNI, Medico, Codigo medico, Fecha, Horario, Motivo y Asistencia.	Intermedio
df	DataFrame con la información de todas las citas registradas.	DataFrame	Generado a partir de listaDictsE.	Intermedio
mapaMedicos	Diccionario que relaciona código de médico con su nombre completo.	dict	Generado desde la lista de médicos registrados.	Intermedio
mapaPacientes	Diccionario que relaciona DNI con el nombre completo del paciente.	dict	Generado desde la lista de pacientes registrados.	Intermedio
opcionG	Opción ingresada por el usuario en el submenú de reportes.	int	Valor entre 1 y 9.	Entrada

5.3. Diseño del menú e interfaces de entrada

- Menú - Ingreso de Opciones

```
def menu():
    print("# SISTEMA DE GESTIÓN DE CONSULTORIO MÉDICO")
    print("[1] Registrar paciente")
    print("[2] Listado Pacientes")
    print("[3] Registrar médico")
    print("[4] Listado médicos")
    print("[5] Crear cita")
    print("[6] Ver citas")
    print("[7] Marcar asistencia")
    print("[8] Generar reporte")
    print("[9] Salir")
    op = validOpcion("Ingresa tu opción: ", 1, 9)
    return op

def validOpcion(mensaje, inf, sup):
    while True:
        try:
            numInt = int(input(mensaje))
            if inf ≤ numInt ≤ sup:
                return numInt
            else:
                print("Opción no válida.\n")
        except ValueError:
            print("EL valor ingresado debe de ser un número.\n")
```

La función menu() muestra en pantalla las diferentes opciones que puede realizar el usuario, como registrar pacientes, registrar médicos, crear citas y generar reportes. Luego, el sistema solicita ingresar una opción para continuar.

Por otro lado, la función validOpcion() valida que la opción ingresada sea un número entero dentro del rango permitido. En caso de que el usuario ingrese un valor incorrecto o un dato no numérico, el sistema muestra un mensaje de error y vuelve a solicitar una opción válida. De esta manera, se garantiza un mejor control y funcionamiento del programa.

- Registrar Paciente

```
def registrarPaciente(self):
    print("\nRegistrando paciente:")
    while True:
        nombresP = input("Nombres: ")
        if nombresP.strip() == "" or any(map(str.isdigit, nombresP)):
            print("No se permiten vacíos ni números.\n")
            continue
        break
    while True:
        apellidosP = input("Apellidos: ")
        if apellidosP.strip() == "" or any(map(str.isdigit, apellidosP)):
            print("No se permiten vacíos ni números.\n")
            continue
        break
    while True:
        dniP = input("DNI (8 dígitos): ")
        repetidoP = False

        if len(dniP) != 8 or not dniP.isdigit():
            print("El DNI debe contener 8 números.\n")
            continue

        for obj in self.__pacientes:
            if obj.dni == dniP:
                repetidoP = True
                break
        if repetidoP:
            print("DNI ya registrado en el sistema. Ingrese otro.\n")
            continue
        break

    while True:
        fechaNacP = (input("Fecha de nacimiento (DDMMAAAA): "))
        if len(fechaNacP) != 8 or not fechaNacP.isdigit():
            print("La fecha debe contener 8 números.\n")
            continue
        dia = int(fechaNacP[:2])
        mes = int(fechaNacP[2:4])
        year = int(fechaNacP[4:])

        if dia < 1 or dia > 31:
            print("Día inválido.\n")
            continue
        elif mes < 1 or mes > 12:
            print("Mes inválido.\n")
            continue
        elif year < 1926 or year > 2026:
            print("Año inválido.\n")
            continue
        else:
            break
    while True:
        lugNacP = (input("Lugar de nacimiento: "))
        if lugNacP.strip() == "" or any(map(str.isdigit, lugNacP)):
            print("No se permiten vacíos ni números.\n")
            continue
        break
    paciente = Paciente(
        nombresP,
        apellidosP,
        dniP,
        fechaNacP,
        lugNacP
    )

    self.__pacientes.append(paciente)
    print("Paciente registrado con éxito!\n")

    listaDictsP = []

    for p in self.__pacientes:
        dictsP = {
            "Nombres": p.nombres,
            "Apellidos": p.apellidos,
            "DNI": p.dni,
            "Fecha nacimiento": p.fechaNac,
            "Lugar nacimiento": p.lugNac
        }
        listaDictsP.append(dictsP)

    dfPacientes = pd.DataFrame(listaDictsP)
    dfPacientes.to_csv("pacientes.csv", sep=";", index=False)
```

- Registrar Médico

```
def registrarMedico(self):
    print("\nRegistrando médico:")
    while True:
        nombresM = input("Nombres: ")
        if nombresM.strip() == "" or any(map(str.isdigit, nombresM)):
            print("No se permiten vacíos ni números.\n")
            continue
        break
    while True:
        apellidosM = input("Apellidos: ")
        if apellidosM.strip() == "" or any(map(str.isdigit, apellidosM)):
            print("No se permiten vacíos ni números.\n")
            continue
        break
    while True:
        codigoM = input("Código de colegiatura médica (6 dígitos): ")
        repetidoM = False

        if len(codigoM) != 6 or not codigoM.isdigit():
            print("El código debe contener 6 números.\n")
            continue

        for obj in self.__medicos:
            if codigoM == obj.codigo:
                repetidoM = True
                break
        if repetidoM:
            print("Código ya registrado en el sistema. Ingrese otro.\n")
            continue
        break
    while True:
        print("¿Posee de alguna especialidad?")
        opcionM = input("Ingrese opcion (Si / No): ")
        opcionM = opcionM.lower()
        if opcionM != "si" and opcionM != "no":
            print("Respuesta inválida. Intentelo de nuevo.\n")
            continue

        if opcionM == "si":
            while True:
                especialM = input("Especialidad: ")
                if especialM.strip() == "" or any(map(str.isdigit, especialM)):
                    print("No se permiten vacíos ni números.\n")
                    continue
                break

        while True:
            codEspecialM = input("Codigo de especialidad: ")
            if len(codEspecialM) != 6 or not codEspecialM.isdigit():
                print("El código de especialidad debe contener 6 números.\n")
                continue
            break
        else:
            especialM = "Médico general"
            codEspecialM = "Desconocido"
            break

    medico = Medico(
        nombresM,
        apellidosM,
        codigoM,
        especialM,
        codEspecialM
    )

    self.__medicos.append(medico)
    print("Médico registrado con éxito!\n")

    listaDictsM = []

    for m in self.__medicos:
        dictsM = {
            "Nombres": m.nombres,
            "Apellidos": m.apellidos,
            "Codigo": m.codigo,
            "Especialidad": m.especial,
            "Codigo especialidad": m.codEspecial
        }
        listaDictsM.append(dictsM)

    dfMedicos = pd.DataFrame(listaDictsM)
    dfMedicos.to_csv("medicos.csv", sep=";", index=False)
```

- Crear Citas Médicas

- Crear cita

```
def crearCita(self):
    print("\nCreando cita:")
    if len(self.__pacientes) == 0 or len(self.__medicos) == 0:
        print("Paciente/Médico no registrado en el sistema.\n")
        return
    dniC = input("Ingrese DNI del paciente: ")
    existe = False

    for p in self.__pacientes:
        if dniC == p.dni:
            existe = True
            break

    if not existe:
        print("DNI no encontrado en el sistema.\n")
        return

    print("\nSeleccione a su médico de preferencia:\n")
    for i, medico in enumerate(self.__medicos, 1):
        print(f"[{i}] {medico.nombres} {medico.apellidos}")
        print(f"Especialidad: {medico.especial}\n")

    while True:
        try:
            opcion = int(input("Elija su opción: "))

            if 1 ≤ opcion ≤ len(self.__medicos):
                opcion -= 1
                break

            print("Opción inválida.\n")

        except ValueError:
            print("Se debe ingresar un número entero.\n")

    medicoC = self.__medicos[opcion].apellidos
    codigoC = self.__medicos[opcion].codigo

    while True:
        fechaC = input("Ingrese Fecha DDMMAAAA (2026-2027): ")
        if len(fechaC) ≠ 8 or not fechaC.isdigit():
            print("La fecha debe contener 8 números.\n")
            continue
        diaC = int(fechaC[:2])
        mesC = int(fechaC[2:4])
        yearC = int(fechaC[4:])

        if diaC < 1 or diaC > 31:
            print("Día inválido.\n")
            continue
        elif mesC < 1 or mesC > 12:
            print("Mes inválido.\n")
            continue
        elif yearC < 2026 or yearC > 2027:
            print("Año inválido.\n")
        else:
            break

    while True:
        horaC = input("Ingrese hora (formato militar): ")
        if len(horaC) ≠ 2 or not horaC.isdigit():
            print("El horario debe contener 2 números.\n")
            continue

        if int(horaC) < 0 or int(horaC) > 23:
            print("Hora inválida.\n")
            continue

        medicOcupadoC = False

        for cita in self.__agenda:
            if (codigoC == cita.codigo and
                fechaC == cita.fecha and
                int(horaC) == int(cita.hora)):
                medicOcupadoC = True
                break

        if medicOcupadoC:
            print("Médico ocupado en ese horario.\n")
            continue

        pacOcupadoC = False

        for cita in self.__agenda:
            if (dniC == cita.dni and
                fechaC == cita.fecha and
                int(horaC) == int(cita.hora)):
                pacOcupadoC = True
                break
```



```

        if pacOcupadoC:
            print("Usted ya tiene una cita en ese horario.\n")
            continue
        break

    while True:

        motivoC = input("Motivo de la consulta (Sea breve): ")

        if motivoC.strip() == "" or any(map(str.isdigit, motivoC)):
            print("No se permiten vacíos ni números.\n")
            continue

        break

    agenda = Agenda(
        dniC,
        medicoC,
        codigoC,
        fechaC,
        horaC,
        motivoC,
    )

    self._agenda.append(agenda)
    print("Cita registrada con éxito!\n")

    listaDictsA = []

    for a in self._agenda:
        dictsA = {
            "DNI": a.dni,
            "Medico": a.medico,
            "Codigo medico": a.codigo,
            "Fecha": a.fecha,
            "Horario": a.hora,
            "Motivo": a.motivo,
            "Asistencia": a.asistio
        }

        listaDictsA.append(dictsA)

    dfAgenda = pd.DataFrame(listaDictsA)
    dfAgenda.to_csv("agenda.csv", sep=";", index=False)

```

- Marcar asistencia

```
def marcarAsistencia(self):
    print(f"\n{'-'*67}")
    print(f"{'CONTROL DE ASISTENCIA':^67}")
    print(f"\n{'-'*67}")

    if len(self.__pacientes) == 0 or len(self.__agenda) == 0:
        print("Paciente/Cita no registrado/a")
        print(f"\n{'-'*67}\n\n")
        return

    dni = input("Ingrese DNI del paciente: ")

    encontrado = False
    tieneCita = False

    for p in self.__pacientes:
        if p.dni == dni:
            encontrado = True
            break

    if not encontrado:
        print("\nDNI no registrado en el sistema")
        print(f"\n{'-'*67}\n\n")
        return

    for cita in self.__agenda:
        if cita.dni == dni:
            tieneCita = True

            print(f"Medico: {cita.medico}")
            print(f"Fecha: {cita.fecha}, Hora: {cita.hora}")
            print(f"Motivo: {cita.motivo}")
            print(f"Estado actual: {cita.asistio}")

            if cita.asistio != "Pendiente":
                print("La asistencia ya fue registrada")
                continue
```

```
while True:
    op = input("Asistio (Si / No): ").lower()
    if op == "si":
        cita.asistio = "si"
        break
    elif op == "no":
        cita.asistio = "no"
        break
    else:
        print("Opción no válida.\n")
        print()

if not tieneCita:
    print("\nEl paciente no tiene citas registradas")
    print(f"\n{'-'*67}\n\n")
    return

print(f"\n{'-'*67}\n\n")

listaDictsA = []

for a in self.__agenda:
    dictsA = {
        "DNI": a.dni,
        "Medico": a.medico,
        "Codigo medico": a.codigo,
        "Fecha": a.fecha,
        "Horario": a.hora,
        "Motivo": a.motivo,
        "Asistencia": a.asistio
    }
    listaDictsA.append(dictsA)

dfAgenda = pd.DataFrame(listaDictsA)
dfAgenda.to_csv("agenda.csv", sep=",", index=False)
```

- Generar ranking

```
def generarReporte(self):
    print("\nREPORTE GENERAL:")
    5.3. Diseño de reportes
    if len(self.__agenda) == 0:
        print("No hay citas registradas")
        print()
        return
```

- GrfAtenxFyM

```
def grfAtenxFyM(self, df, mapaMedicos):
    print("\nATENCIONES POR FECHA Y MEDICO")

    atendidas = df[df["Asistencia"] == "si"]

    if atendidas.empty:
        print("No hay atenciones registradas para graficar.")
        return

    for codigo in atendidas["Codigo medico"].unique():
        datos = atendidas[atendidas["Codigo medico"] == codigo]
        resumen = datos.groupby("FechaDT").size()
        nombre = mapaMedicos.get(codigo, codigo)
        plt.plot(resumen.index, resumen.values, label=nombre, linewidth=2)

    plt.title("Atenciones por Fecha y por Médico")
    plt.xlabel("Fecha")
    plt.ylabel("Cantidad de Atenciones")
    plt.legend()
    plt.grid(True, color="#33A5FF")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

- GrfAtenxFyP

```
def grfAtenxFyP(self, df, mapaPacientes):
    print("\nATENCIONES POR FECHA Y PACIENTE")

    atendidas = df[df["Asistencia"] == "si"]

    if atendidas.empty:
        print("No hay atenciones registradas para graficar.")
        return

    pacientesTop = atendidas["DNI"].value_counts().head(5).index

    for dni in pacientesTop:
        datos = atendidas[atendidas["DNI"] == dni]
        resumen = datos.groupby("FechaDT").size()
        nombre = mapaPacientes.get(dni, dni)
        plt.plot(resumen.index, resumen.values, label=nombre, linewidth=2)

    plt.title("Atenciones por Fecha y por Paciente (Top 5)")
    plt.xlabel("Fecha")
    plt.ylabel("Cantidad de Atenciones")
    plt.legend()
    plt.grid(True, color="#FF4433")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

- GrfRankingM

```
def grfRankingM(self, df, mapaMedicos):
    print("\nRANKING DE MEDICOS POR CITAS ATENDIDAS")

    medicOrd = self.generarRanking(df, "Codigo medico")

    if medicOrd.empty:
        print("No hay atenciones registradas para graficar.")
        return

    nombres = [mapaMedicos.get(codigo, codigo) for codigo in medicOrd.index]

    plt.bar(nombres, medicOrd.values, color="#33FF36", width=0.5)
    plt.title("Ranking de Médicos por Citas Atendidas")
    plt.xlabel("Médico")
    plt.ylabel("Citas Atendidas")
    plt.grid(True, color="#FF334E")
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.show()
```

- GrfPacntMasCit

```
def grfPacntMasCit(self, df, mapaPacientes):
    print("\nPACIENTES CON MAS CITAS")

    pacOrd = self.generarRanking(df, "DNI").head(10)

    if pacOrd.empty:
        print("No hay atenciones registradas para graficar.")
        return

    nombres = [mapaPacientes.get(dni, dni) for dni in pacOrd.index]

    plt.bar(nombres, pacOrd.values, color="#FFAA33", width=0.5)
    plt.title("Pacientes con Más Citas (Top 10)")
    plt.xlabel("Paciente")
    plt.ylabel("Cantidad de Citas Atendidas")
    plt.grid(True, color="#6333FF")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

- GrfInasist

```
def grfInasist(self, df, mapaMedicos):
    print("\nINASISTENCIAS POR MEDICO")

    inasistentes = df[df["Asistencia"] == "no"]

    if inasistentes.empty:
        print("No hay inasistencias registradas para graficar.")
        return

    conteo = inasistentes.groupby("Codigo medico").size().sort_values(ascending=False)
    nombres = [mapaMedicos.get(codigo, codigo) for codigo in conteo.index]

    plt.bar(nombres, conteo.values, color="#FF3355", width=0.5)
    plt.title("Inasistencias por Médico")
    plt.xlabel("Médico")
    plt.ylabel("Cantidad de Inasistencias")
    plt.grid(True, color="#33A5FF")
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.show()
```

- GrfAtenVsCanc

```
def grfAtenVsCanc(self, df):
    print("\nCITAS ATENDIDAS VS CANCELADAS")

    atendidasN = (df["Asistencia"] == "si").sum()
    canceladasN = (df["Asistencia"] == "no").sum()

    if atendidasN == 0 and canceladasN == 0:
        print("No hay citas con asistencia registrada para graficar.")
        return

    plt.pie(
        [atendidasN, canceladasN],
        labels=["Atendidas", "Canceladas"],
        autopct="%0.1f%%",
        colors=["#33FF36", "#FF3355"]
    )
    plt.title("Citas Atendidas vs Canceladas")
    plt.axis("equal")
    plt.show()
```


- GrfCitsxEspec

```
def grfCitsxEspec(self, df):
    print("\nCITAS POR ESPECIALIDAD")

    mapaEspecialidad = {m.codigo: m.especial for m in self.__medicos}
    df["Especialidad"] = df["Codigo medico"].map(mapaEspecialidad)

    conteo = df["Especialidad"].value_counts()

    if conteo.empty:
        print("No hay citas registradas para graficar.")
        return

    plt.bar(conteo.index, conteo.values, color="#33C4FF", width=0.5)
    plt.title("Citas por Especialidad")
    plt.xlabel("Especialidad")
    plt.ylabel("Cantidad de Citas")
    plt.grid(True, color="#FF8633")
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.show()
```

- GrfRnkgMxEspc

```
def grfRnkgMxEspc(self, df):
    print("\nRANKING DE MEDICOS POR ESPECIALIDAD")

    especialidadesR = []

    for medico in self.__medicos:
        if medico.especial not in especialidadesR:
            especialidadesR.append(medico.especial)

    for especialidad in especialidadesR:
        medicosEsp = [m for m in self.__medicos if m.especial == especialidad]

        nombres = []
        cantidades = []

        for medico in medicosEsp:
            contadorR = (df["Codigo medico"] == medico.codigo).sum()
            nombres.append(f"{medico.nombres} {medico.apellidos}")
            cantidades.append(contadorR)

    plt.bar(nombres, cantidades, color="#A533FF", width=0.5)
    plt.title(f"Ranking de Médicos - {especialidad}")
    plt.xlabel("Médico")
    plt.ylabel("Cantidad de Citas")
    plt.grid(True, color="#33FF9E")
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.show()
```

- Generar reporte

```
def generarReporte(self):
    print("\nREPORTE GENERAL:")
    if len(self.__agenda) == 0:
        print("No hay citas registradas")
        print()
        return
    print("\n# ATENCIONES REALIZADAS FECHA/MÉDICO")
    print(f"{'\n#':<5}{'Fecha':<15}{'Hora':<10}{'Codigo':<15}")

    contador1 = 1
    for cita in self.__agenda:
        if cita.asistio == "si":
            print(
                f"{contador1:<5}"
                f"{cita.fecha:<15}"
                f"{str(cita.hora) + 'h':<10}"
                f"{cita.codigo:<15}"
            )
            contador1 += 1

    print("\n# ATENCIONES REALIZADAS FECHA/PACIENTE")
    print(f"{'\n#':<5}{'Fecha':<15}{'Hora':<10}{'DNI':<15}")

    contador2 = 1
    for cita in self.__agenda:
        if cita.asistio == "si":
            print(
                f"{contador2:<5}"
                f"{cita.fecha:<15}"
                f"{str(cita.hora) + 'h':<10}"
                f"{cita.dni:<15}"
            )
            contador2 += 1

    listaDictsE = []
    for a in self.__agenda:
        dictsE = {
            "DNI": a.dni,
            "Medico": a.medico,
            "Codigo medico": a.codigo,
            "Fecha": a.fecha,
            "Horario": a.hora,
            "Motivo": a.motivo,
            "Asistencia": a.asistio
        }
        listaDictsE.append(dictsE)

    df = pd.DataFrame(listaDictsE)

    medicOrd = self.generarRanking(df, "Codigo medico")

    print("\n# RANKING DE MÉDICOS")
    print(f"{'\n#':<5}{'Puesto':<10}{'Codigo':<20}{'Atenciones':<15}")

    puesto1 = 1
    for codigo, repite in medicOrd.items():
        print(f"{puesto1:<10}{codigo:<20}{repite:<15}")
        puesto1 += 1

    pacOrd = self.generarRanking(df, "DNI")

    print("\n# RANKING DE PACIENTES")
    print(f"{'\n#':<5}{'Puesto':<10}{'DNI':<20}{'Atenciones':<15}")

    puesto2 = 1
    for dni, repite in pacOrd.items():
        print(f"{puesto2:<10}{dni:<20}{repite:<15}")
        puesto2 += 1

    print("\n# CITAS ATENDIDAS VS CANCELADAS")

    atendidasN = 0
    canceladasN = 0

    for cita in self.__agenda:
        if cita.asistio == "si":
            atendidasN += 1
        elif cita.asistio == "no":
            canceladasN += 1

    print(f"Citas atendidas: {atendidasN}")
    print(f"Citas canceladas: {canceladasN}")

    print("\n# CITAS POR ESPECIALIDAD")

    especialidadesE = []

    for medico in self.__medicos:
        if medico.especial not in especialidadesE:
            especialidadesE.append(medico.especial)
```

5.4. Diseño de reportes

- Función main

```
def main():
    sistema1 = Sistema()
    while True:
        opcion = menu()

        match opcion:
            case 1:
                sistema1.registrarPaciente()
            case 2:
                sistema1.listarPacientes()
            case 3:
                sistema1.registrarMedico()
            case 4:
                sistema1.listarMedicos()
            case 5:
                sistema1.crearCita()
            case 6:
                sistema1.listarCitas()
            case 7:
                sistema1.marcarAsistencia()
            case 8:
                sistema1.generarReporte()
            case 9:
                print("Muchas gracias por confiar en nosotros!")
                print("Saliendo del sistema...")
                break

# pp
main()
```

- Load pacientes

```
def loadPacientes(self):
    try:
        dfP = pd.read_csv(
            "pacientes.csv",
            sep=";",
            dtype={"DNI": str, "Fecha nacimiento": str}
        )
    except FileNotFoundError:
        return

    for i in range(len(dfP)):
        paciente = Paciente(
            dfP.iloc[i]["Nombres"],
            dfP.iloc[i]["Apellidos"],
            dfP.iloc[i]["DNI"],
            dfP.iloc[i]["Fecha nacimiento"],
            dfP.iloc[i]["Lugar nacimiento"]
        )
        self.__pacientes.append(paciente)
```


- Load Medicos

```
def loadMedicos(self):
    try:
        dfM = pd.read_csv(
            "medicos.csv",
            sep=";",
            dtype={"Codigo": str, "Codigo especialidad": str}
        )
    except FileNotFoundError:
        return

    for i in range(len(dfM)):
        medico = Medico(
            dfM.iloc[i]["Nombres"],
            dfM.iloc[i]["Apellidos"],
            dfM.iloc[i]["Codigo"],
            dfM.iloc[i]["Especialidad"],
            dfM.iloc[i]["Codigo especialidad"]
        )
        self.__medicos.append(medico)
```

- Load agenda

```
def loadAgenda(self):
    try:
        dfA = pd.read_csv(
            "agenda.csv",
            sep=";",
            dtype=str
        )
    except FileNotFoundError:
        return

    for i in range(len(dfA)):
        agenda = Agenda(
            dfA.iloc[i]["DNI"],
            dfA.iloc[i]["Medico"],
            dfA.iloc[i]["Codigo medico"],
            dfA.iloc[i]["Fecha"],
            dfA.iloc[i]["Horario"],
            dfA.iloc[i]["Motivo"],
            dfA.iloc[i]["Asistencia"]
        )
        self.__agenda.append(agenda)
```

- Listado pacientes

```
def listarPacientes(self):
    print("\nLISTA DE PACIENTES REGISTRADOS EN EL SISTEMA: ")
    try:
        df = pd.read_csv("pacientes.csv", sep=",", dtype=str)
        print(df)
        print()
    except FileNotFoundError:
        print("Aún no hay pacientes registrados.\n")
```

- Listado médico

```
def listarMedicos(self):
    print("\nLISTA DE MÉDICOS REGISTRADOS EN EL SISTEMA: ")
    try:
        df = pd.read_csv("medicos.csv", sep=",", dtype=str)
        print(df)
        print()
    except FileNotFoundError:
        print("Aún no hay médicos registrados.\n")
```

- Listar cita

```
def listarCitas(self):
    print("\nLISTA DE CITAS REGISTRADAS EN EL SISTEMA: ")
    try:
        df = pd.read_csv("agenda.csv", sep=",", dtype=str)
        print(df)
        print()
    except FileNotFoundError:
        print("Aún no hay citas registradas.\n")
```

5.4. Manual de Usuario

- Menú Principal

```
# SISTEMA DE GESTIÓN DE CONSULTORIO MÉDICO
[1] Registrar paciente
[2] Listado Pacientes
[3] Registrar médico
[4] Listado médicos
[5] Crear cita
[6] Ver citas
[7] Marcar asistencia
[8] Generar reporte
[9] Salir
Ingresa tu opción:
```

- Registrar paciente

```
Ingresa tu opción: 1

Registrando paciente:
Nombres: Diego Ricardo
Apellidos: Aldave Ocrosoma
DNI (8 dígitos): 87234534
Fecha de nacimiento (DDMMAAAA): 22122007
Lugar de nacimiento: Lima
¡Paciente registrado con éxito!
```

- Listado Pacientes

Ingresa tu opción: 2

LISTA DE PACIENTES REGISTRADOS EN EL SISTEMA:

	<i>Nombres</i>	<i>Apellidos</i>	<i>DNI</i>	<i>Fecha nacimiento</i>	<i>Lugar nacimiento</i>
0	Jayron Fabricio	Mercado Torres	70703768	28052007	Chaclacayo
1	Maria Fernanda	Rojas Quispe	71234567	15031999	Lima
2	Carlos Andres	Huaman Soto	72345678	03071995	Arequipa
3	Lucia Belen	Castillo Vega	73456789	22111988	Trujillo
4	Diego Alonso	Ramos Flores	74567890	10012001	Cusco
5	Valeria Sofia	Chavez Diaz	75678901	30092000	Piura
6	Renzo Gabriel	Salazar Nunez	76789012	05041992	Chiclayo
7	Camila Alejandra	Torres Medina	77890123	18081997	Ica
8	Bruno Sebastian	Guerrero Paz	78901234	25122003	Tacna
9	Antonella Nicole	Vargas Leon	79012345	12061994	Huancayo
10	Diego Ricardo	Aldave Ocrospoma	87234534	22122007	Lima

- Registrar médico

Ingresa tu opción: 3

Registrando médico:
Nombres: Thiago Alexander
Apellidos: Cabrera Garcia
Código de colegiatura médica (6 dígitos): 110807
¿Posee de alguna especialidad?
Ingrese opcion (Si / No): no
¡Médico registrado con éxito!

- Listado médicos

Ingresa tu opción: 4

LISTA DE MÉDICOS REGISTRADOS EN EL SISTEMA:

	<i>Nombres</i>	<i>Apellidos</i>	<i>Codigo</i>	<i>Especialidad</i>	<i>Codigo especialidad</i>
0	Jose Luis	Fernandez Rios	100001	Cardiologia	200001
1	Ana Patricia	Salinas Cordova	100002	Pediatrica	200002
2	Ricardo	Mendoza Alva	100003	Medico General	Desconocido
3	Thiago Alexander	Cabrera Garcia	110807	Médico General	Desconocido

- Crear Cita

```

Ingresa tu opción: 5

Creando cita:
Ingresa DNI del paciente: 87234534

Seleccione a su médico de preferencia:

[1] Jose Luis Fernandez Rios
Especialidad: Cardiologia

[2] Ana Patricia Salinas Cordova
Especialidad: Pediatria

[3] Ricardo Mendoza Alva
Especialidad: Medico General

[4] Thiago Alexander Cabrera Garcia
Especialidad: Médico General

Elija su opción: 4
Ingresa Fecha DDMMAAAA (2026-2027): 100726
La fecha debe contener 8 números.

Ingresa Fecha DDMMAAAA (2026-2027): 10072026
Ingresa hora (formato militar): 1500
El horario debe contener 2 números.

Ingresa hora (formato militar): 16
Motivo de la consulta (Sea breve): Dolor intenso del abdomen
¡Cita registrada con éxito!

```

- Ver cita

```

Ingresa tu opción: 6

LISTA DE CITAS REGISTRADAS EN EL SISTEMA:

```

	DNI	Medico	Codigo medico	Fecha	Horario	Motivo	Asistencia
0	70703768	Fernandez Rios	100001	10072026	09	Chequeo de rutina	si
1	71234567	Salinas Cordova	100002	10072026	10	Control de vacunas	no
2	72345678	Mendoza Alva	100003	10072026	11	Dolor de cabeza	si
3	73456789	Fernandez Rios	100001	11072026	09	Dolor en el pecho	no
4	74567890	Salinas Cordova	100002	11072026	14	Fiebre alta	no
5	75678901	Mendoza Alva	100003	11072026	16	Gripe	no
6	76789012	Fernandez Rios	100001	12072026	08	Presion alta	si
7	77890123	Salinas Cordova	100002	15072026	10	Control mensual	si
8	78901234	Mendoza Alva	100003	20072026	17	Consulta general	si
9	79012345	Fernandez Rios	100001	25072026	13	Seguimiento post cirugia	si
10	87234534	Cabrera Garcia	110007	10072026	16	Dolor intenso del abdomen	Pendiente

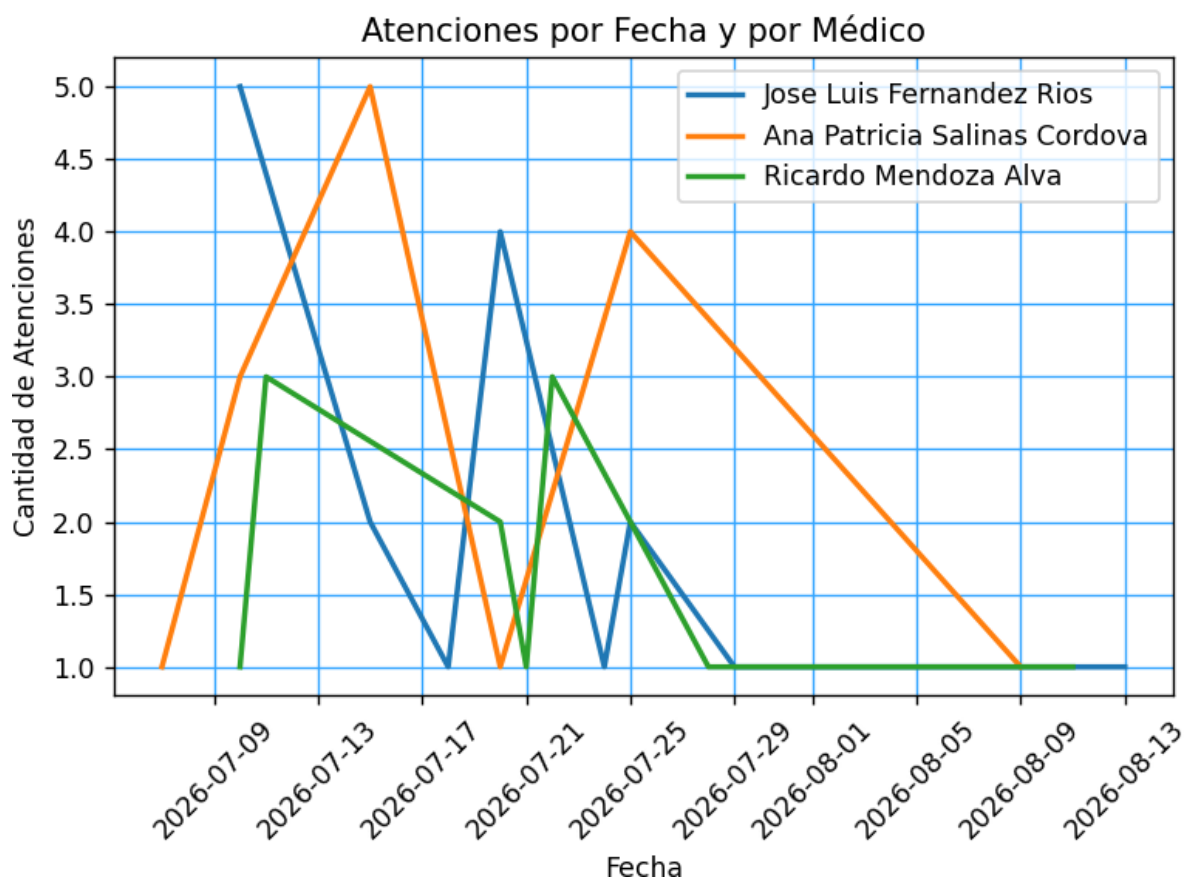
- Marcar asistencia

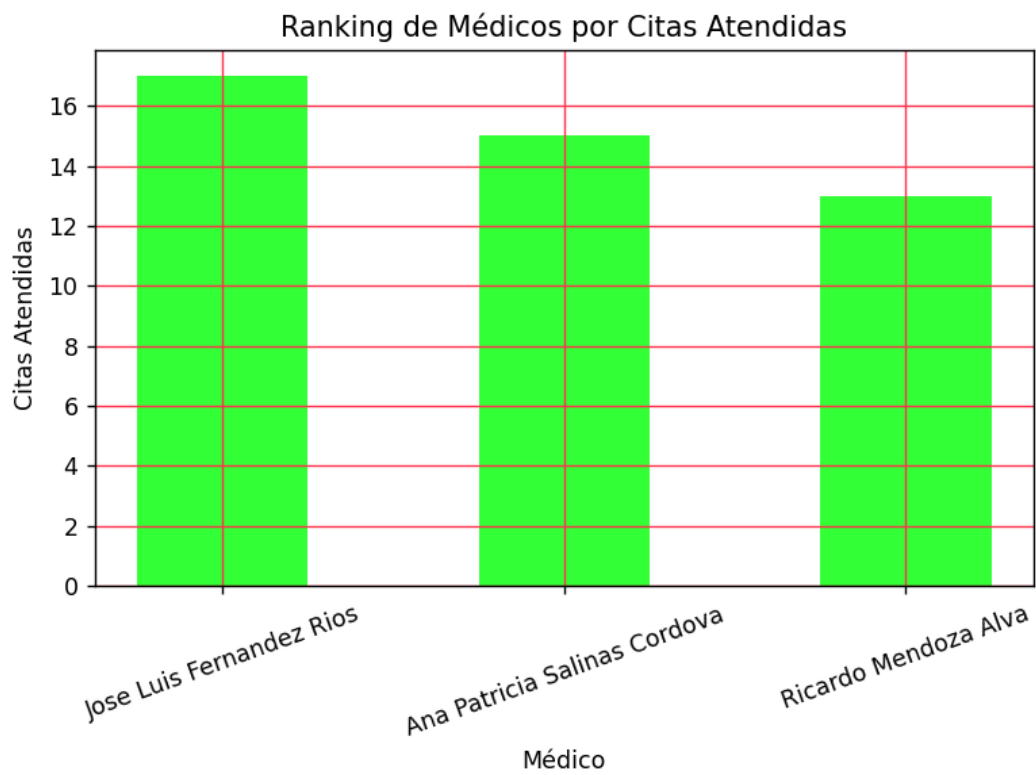
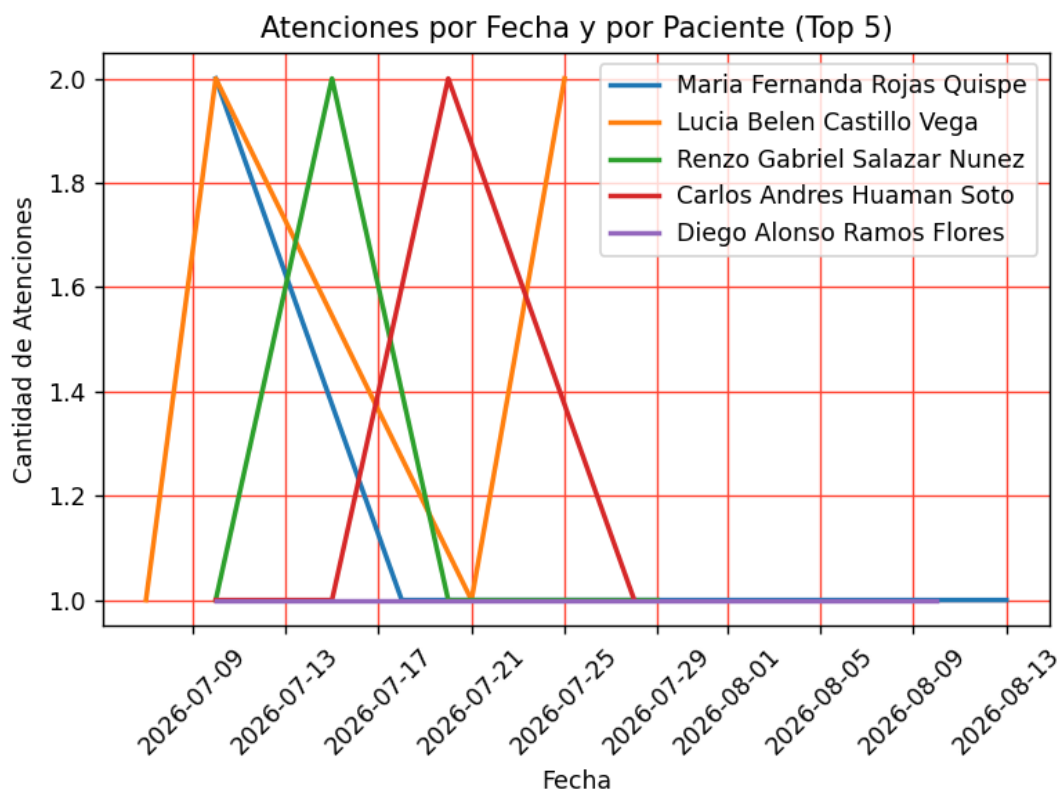
Ingresa tu opción: 7

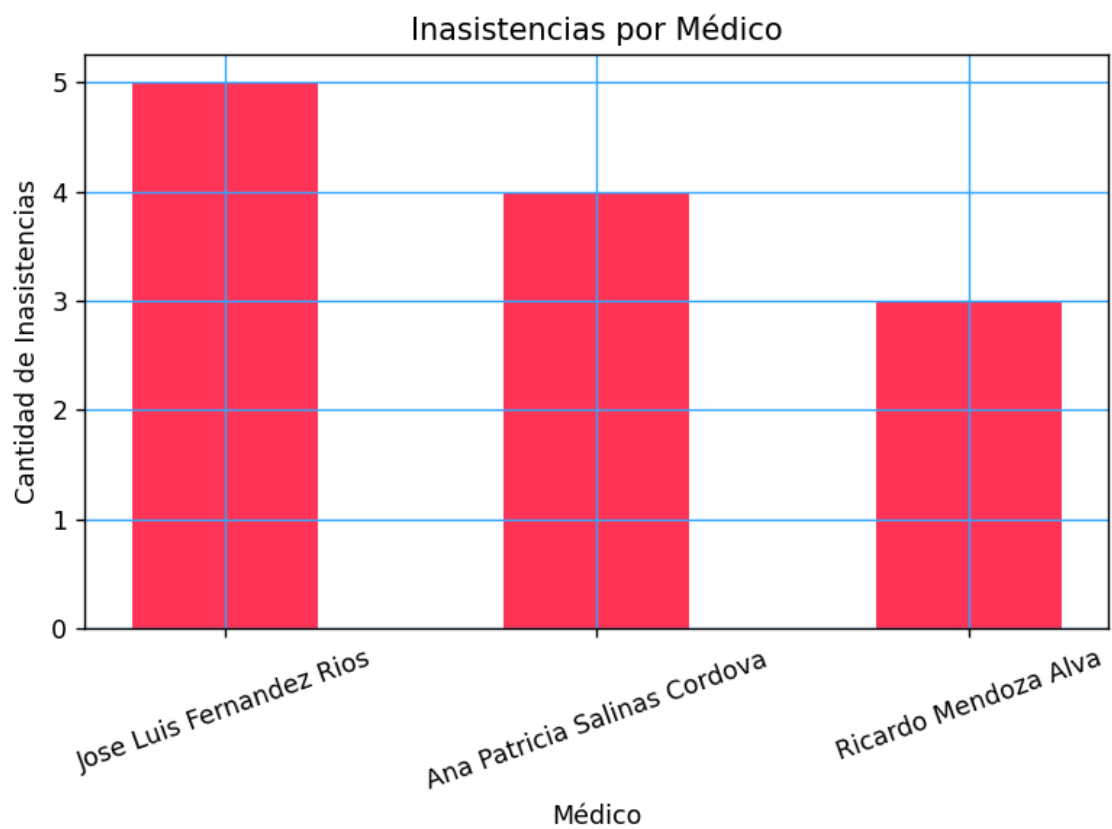
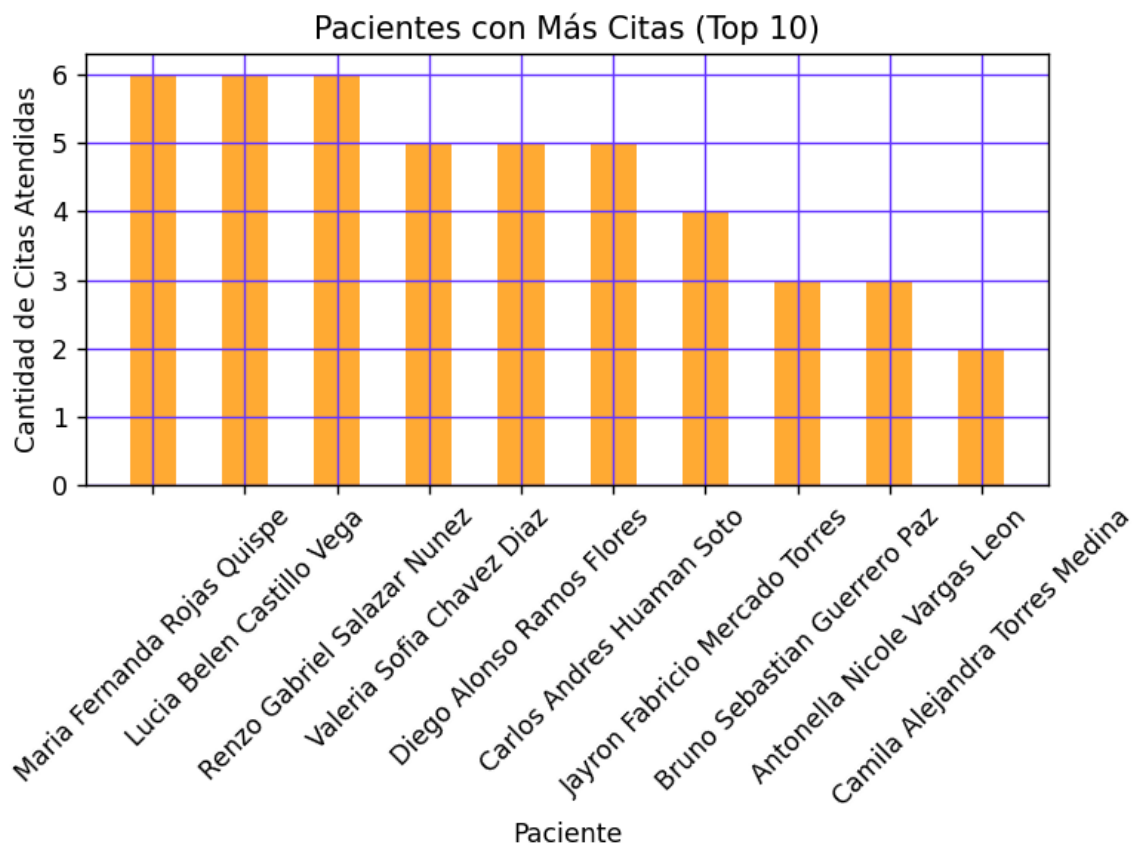
CONTROL DE ASISTENCIA

Ingresa DNI del paciente: 87234534
Médico: Cabrera García
Fecha: 10072026, Hora: 16
Motivo: Dolor intenso del abdomen
Estado actual: Pendiente
Asistio (Si / No): si

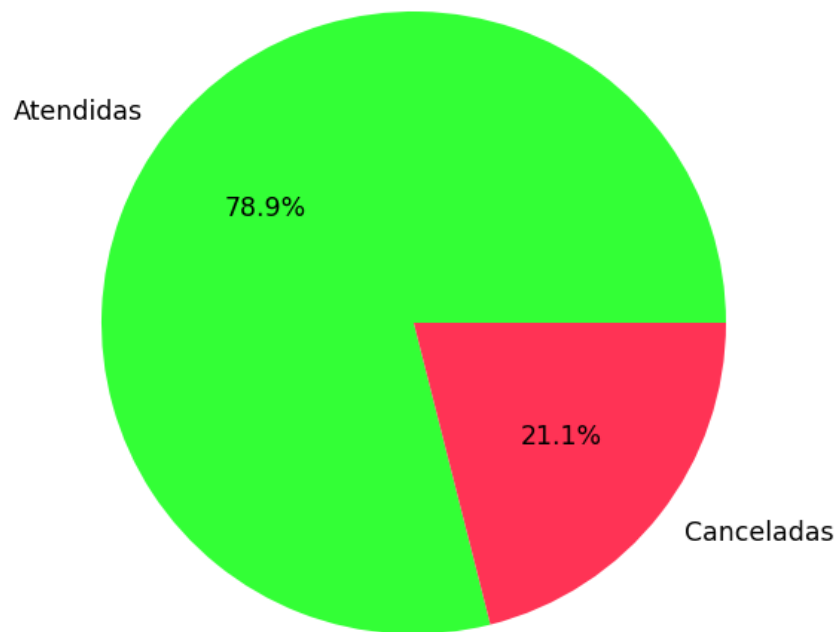
- Generar reporte



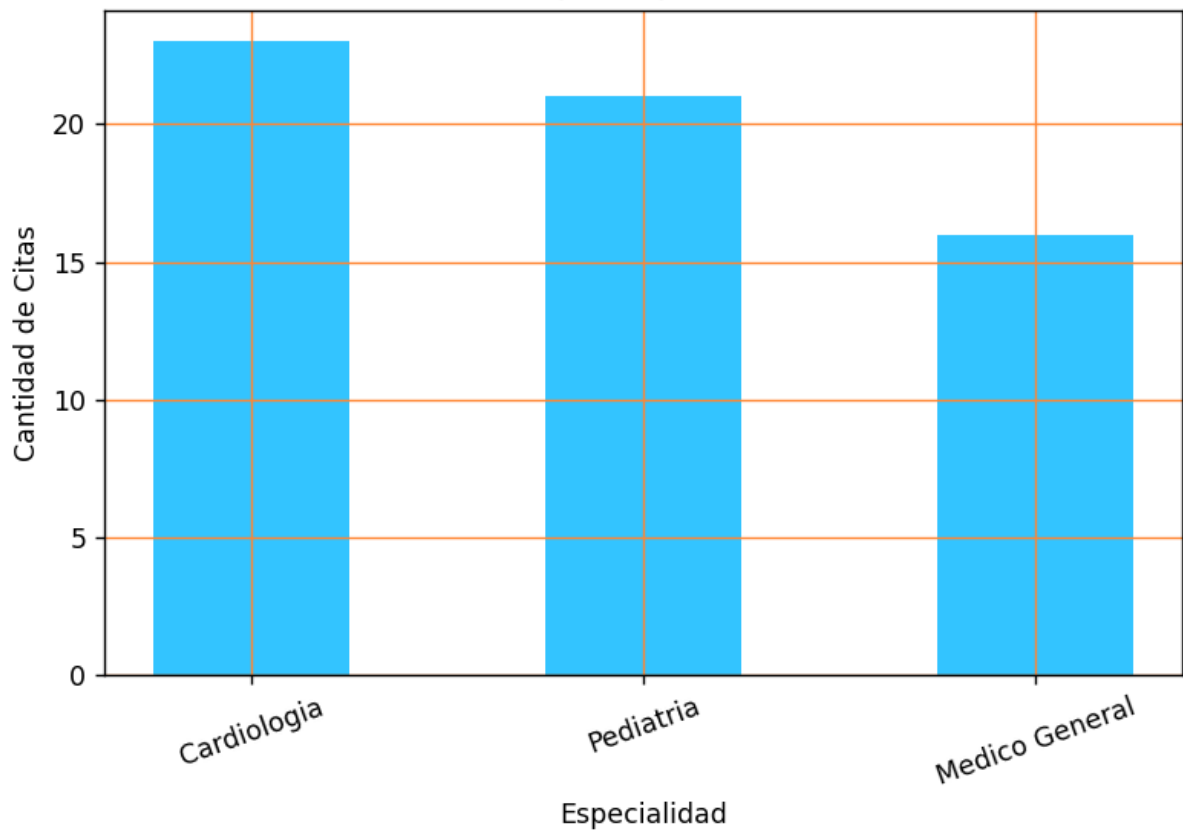


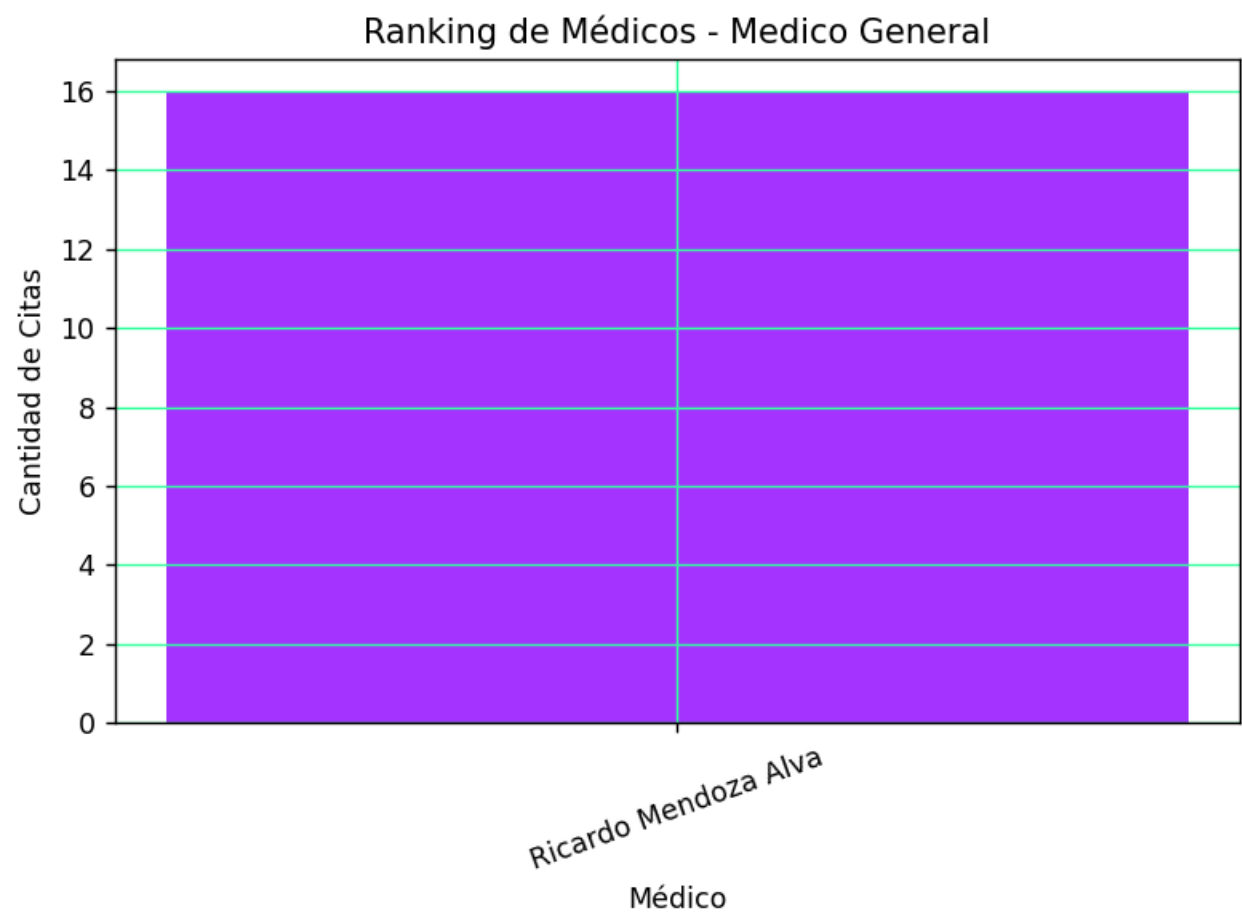
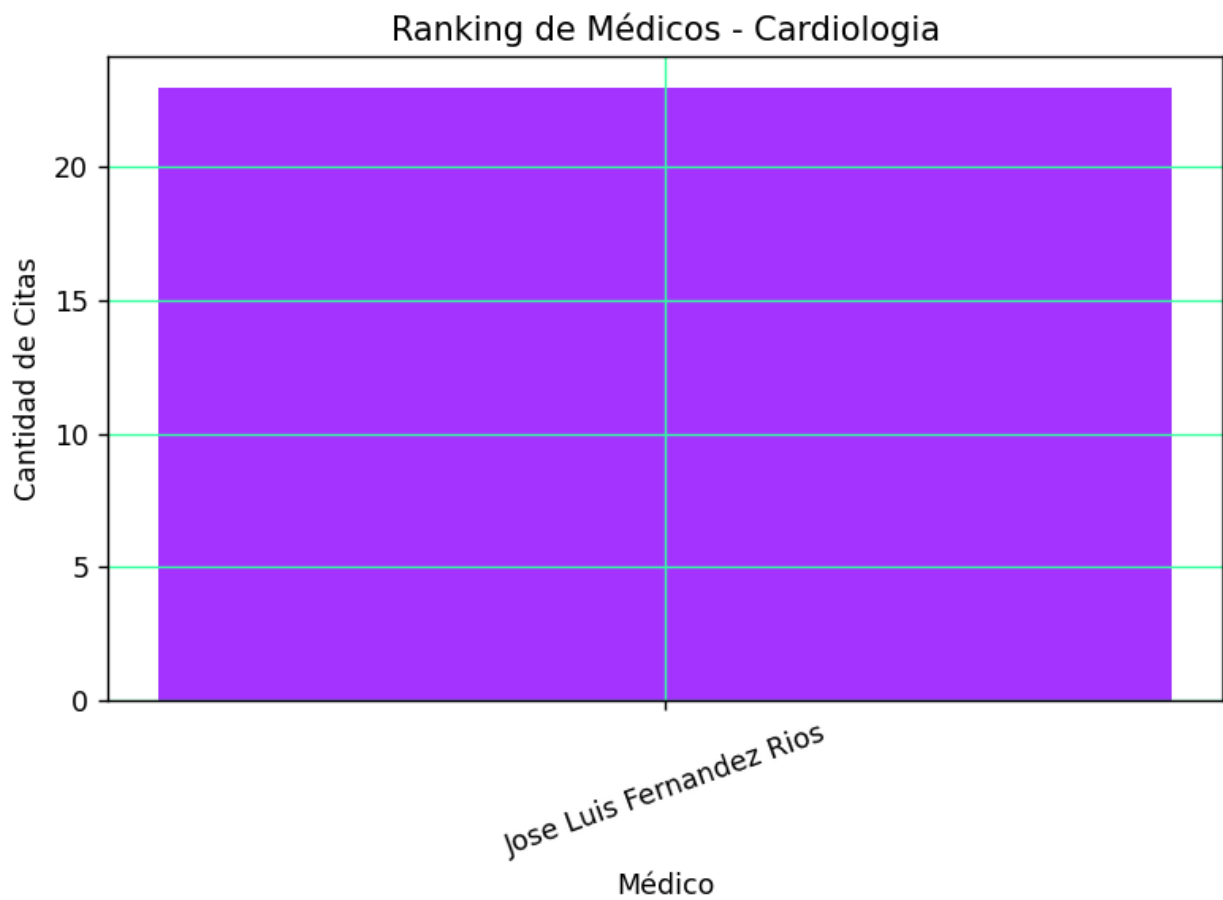


Citas Atendidas vs Canceladas

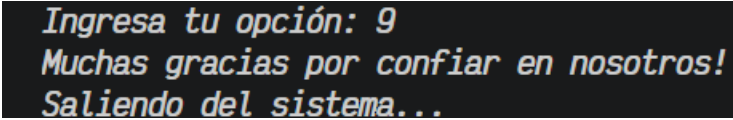


Citas por Especialidad





- Salir



```
Ingresa tu opción: 9
Muchas gracias por confiar en nosotros!
Saliendo del sistema...
```

5.5. Estándar de nomenclatura

En este proyecto se adopta la convención “camelCase” para nombrar variables, listas, tuplas y funciones, con el objetivo de mantener uniformidad y facilitar la lectura del código. Este estilo consiste en escribir los identificadores sin espacios, iniciando con letra minúscula y utilizando mayúsculas para separar cada palabra adicional. Su uso permite distinguir fácilmente los nombres compuestos, mejora la comprensión del programa y favorece el mantenimiento y la escalabilidad del sistema.

6. Conclusiones

Aldave Ocrospoma Diego Ricardo:

Con el desarrollo de este proyecto, logré poner en práctica los temas avanzados en las sesiones de clase, lo que me permitió consolidar el aprendizaje adquirido. Asimismo, la experiencia de diseñar un flujo automatizado para el control de registros fue de gran utilidad para entender cómo estructurar soluciones informáticas eficientes que puedan aplicarse en futuros proyectos.

Mercado Torres, Jayron Fabricio:

El desarrollo de este proyecto me permitió aplicar las herramientas clave del lenguaje Python para la automatización de flujos y la persistencia de datos. Asimismo, sirvió como un excelente recurso para consolidar y tener más presentes aquellos temas que requerían reforzamiento con el avance de las semanas. Finalmente, la experiencia del trabajo grupal contribuyó al fortalecimiento de mis capacidades de coordinación colectiva, logrando un diseño de software conjunto basado en la toma de decisiones consensuadas.

Rodriguez Quispe, Dario Francisco:

El desarrollo de este proyecto me permitió aplicar los conocimientos adquiridos en programación a través de un modelo basado en clases y objetos en Python. Además,

me brindó una mejor comprensión sobre cómo estructurar sistemas orientados a la organización y administración de información en un centro de salud, contribuyendo a optimizar el control de los registros de manera eficiente.

Asimismo, el trabajo conjunto permitió fortalecer habilidades de análisis lógico, resolución de problemas y diseño de soluciones aplicadas a situaciones reales. Finalmente, se evidenció el impacto positivo de la tecnología en el área médica, demostrando cómo la automatización de procesos mejora la gestión de la información dentro del entorno de asistencia.

Salcedo Ramírez, Yasuri Yamile:

A través de la elaboración de este proyecto se logró comprender mejor el funcionamiento de un sistema de gestión médica desarrollado en Python, aplicando validaciones, registros y control de información mediante un modelo estructurado. El desarrollo de la arquitectura del programa permitió mejorar la capacidad para plantear soluciones lógicas y organizar los datos de manera eficiente.

También se identificó la importancia de verificar y controlar correctamente la información ingresada por el usuario para evitar inconsistencias dentro del sistema. De esta manera, el proyecto permitió reforzar habilidades relacionadas con la programación y la resolución de problemas en situaciones similares a un entorno real.

7. Recomendaciones

Aldave Ocrospoma Diego Ricardo:

Recomiendo que los futuros alumnos de este curso practiquen constantemente la lógica de programación y que sean conscientes del funcionamiento de las funciones del código antes de implementarlas, así como es importante la colaboración de todos los miembros del equipo para poder cubrir las perspectivas de todos, asegurándose que sea necesario.

Mercado Torres, Jayron Fabricio:

Mis recomendaciones para los futuros segundo ciclo y que estén cursando POO, sería que no sean confiados, estudien mínimo 2 horas diarias del curso, resolviendo ejercicios ya que de esa manera empiezas a tener más lógica de programación, ya que una vez agarras la lógica ya lo demás se te facilita entenderlo en cierta manera, también

recomendaría que, si tienen dudas, pregunten, la vergüenza es momentánea, el conocimiento adquirido, eterno.

Rodriguez Quispe, Dario Francisco:

Recomiendo a los futuros estudiantes practicar constantemente la lógica de programación antes de desarrollar proyectos grandes, ya que esto facilita la comprensión y solución de problemas durante la elaboración del programa.

Profundizar en el lenguaje de Python para tener un máximo conocimiento y reforzar el uso de estructuras de datos como listas, diccionarios y tuplas, la programación orientada a objetos, como la librería pandas y matplotlib para el manejo de datos puesto que son fundamentales para almacenar y gestionar información en distintos tipos de sistemas.

Salcedo Ramírez, Yasuri Yamile:

Recomiendo a los futuros estudiantes practicar constantemente programación en Python para mejorar la lógica y el desarrollo de sistemas. Además, empezar a crear proyectos personales pequeños para reforzar la práctica y adquirir mayor experiencia en el desarrollo de programas.

Por último, nunca dejar de aprender y explorar nuevas herramientas de Python para seguir mejorando las habilidades de programación.

8. Glosario

- **Sistema de gestión médica:** Programa que ayuda a organizar y registrar información de pacientes, médicos y citas.
- **Validación de datos:** Proceso que verifica la información ingresada por el usuario antes de guardarla en el sistema.
- **Base de datos temporal:** Información almacenada momentáneamente mientras se ejecuta el programa.
- **Diccionario en Python:** Estructura para guardar datos relacionados entre sí, por ejemplo, nombre y DNI.
- **Lista dinámica:** Conjunto de datos que puede aumentar o modificarse mientras funciona el sistema.

- **Interfaz de consola:** Forma de interacción entre el usuario y el programa.
- **Estructura condicional:** Instrucción del programa que permite tomar decisiones dependiendo de una condición.
- **Estructura repetitiva:** Bloque de código que repite una acción varias veces hasta cumplir una condición.
- **Función:** Bloque de instrucciones diseñados para realizar una tarea en específico dentro del programa.
- **Parámetro:** Dato que se envía a una función para ejecutar una determinada acción.
- **Variable:** Espacio utilizado para almacenar información.
- **Tupla:** Tipo de estructura de datos que permite guardar varios valores juntos en una sola variable.

9. Bibliografía

García, J. (2017). *Python como primer lenguaje de programación textual en la enseñanza secundaria*. Revista Iberoamericana de Educación, 18(2), 147-162. <https://www.redalyc.org/pdf/5355/535554766009.pdf>

Ortiz Zambrano, J. A., Sangacha Tapia, L. M., & Alarcón Santillán, J. A. (2017). *Importancia de la programación en la formación de los ingenieros de sistemas computacionales*. Opuntia Brava, 9(4). Universidad de Las Tunas. <https://www.redalyc.org/pdf/5738/573867429003.pdf>

Recharte Urrutia, P. E. (2023). *Sistema informático de historias clínicas para optimizar la atención a pacientes del Centro de Salud Cono Norte Ayaviri, 2021* [Tesis de licenciatura, Universidad Nacional del Altiplano]. Repositorio institucional. <https://repositorio.unap.edu.pe/items/096ba6d2-2d1e-45d4-ab79-4cc861566733>

Villafuerte Salas, C. V. y Villanueva Yana, D. P. (2020). *Sistema de gestión de la información de las historias clínicas en el Hospital PNP Augusto B. Leguía* [Tesis de maestría, Pontificia Universidad Católica del Perú]. Repositorio institucional. <https://tesis.pucp.edu.pe/items/e96e977b-58d4-44c4-969c-2085d9b81509>

10. Participación

Tabla 7

Porcentaje de participación

Integrantes:	Porcentaje de participación
Aldave Ocrosoma Diego Ricardo	100%
Mercado Torres, Jayron Fabricio	100%
Rodriguez Quispe, Dario Francisco	100%
Salcedo Ramírez, Yasuri Yamile	100%

